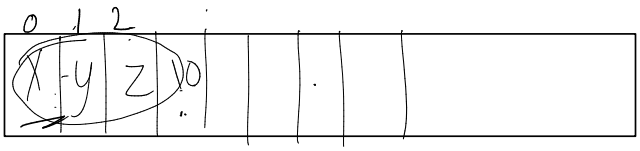



S1

```
for (i=l1; i>=posn; i--)
    s1[i+l2] = s1[i];
```

```
for (i=0; s2[i]; i++)
    s1[posn+i] = s2[i];
```



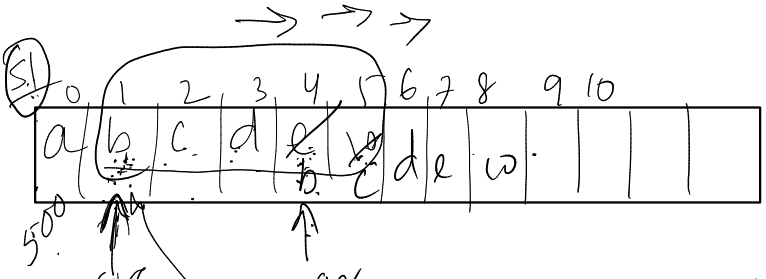

step1: move s1+posn string towards right len2 bytes

step2: copy s2 to s1+posn (not the \0)

len1 = 5
len2 = 3
posn = 1

```
{
gets(s1); gets(s2);  l1 = strlen(s1); l2 = strlen(s2);
strMove(s1+posn+l2, s1+posn, strlen(s1+posn)+1);
}
```

mem  tar.  src  5




l2 = 3

pom 21

bcde\0

```
void strMove(char *tar, char *src, int n)
{
    tar[8] = src[5]
    tar[7] = src[4]
    tar[6] = src[3]
    [5] = [2]
    [4] = [1]
}
```

504  501  int n  5

memmove(tar, src, n bytes)

s1+posn+l2, s1+posn, strlen(s1+posn)+1

strncpy(s1+posn, s2, l2);

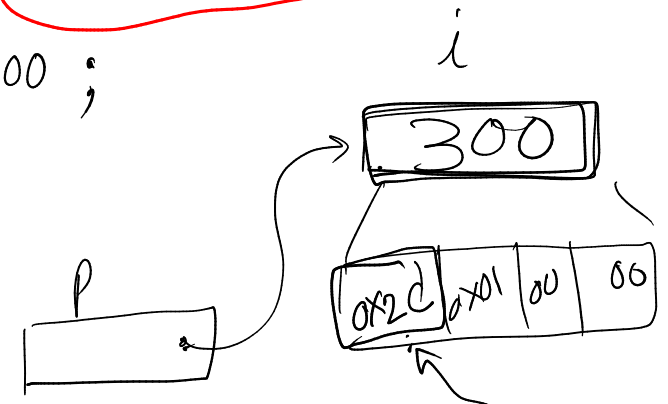
→ void *memmove(void *tar, const void *src, size_t len);

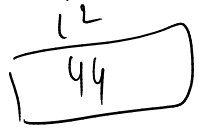
→ char *strncpy(char *tar, const char *src, size_t len);

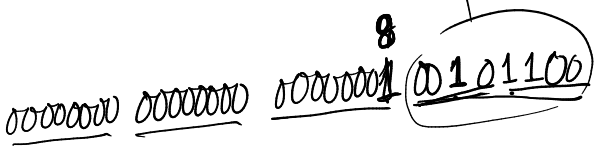
# Generic Pointer / void *

MUST BE Typecasted when dereferenced

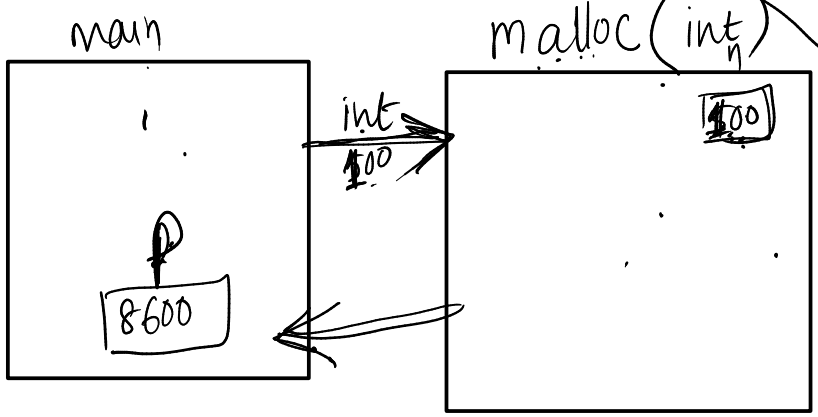
```
{
    char *p;
    void *q;
    int i = 300;
    int i2;
    p = &i;
    i2 = *p;
    // i2 = *(int *)p;
    q = &i;
    i2 = *q; // Syntax Error
    i2 = *(int *)q;
```

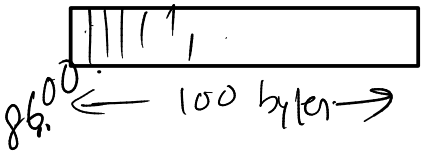



```
{
    void v;
    int x;
    float y;
    char z;
}
```

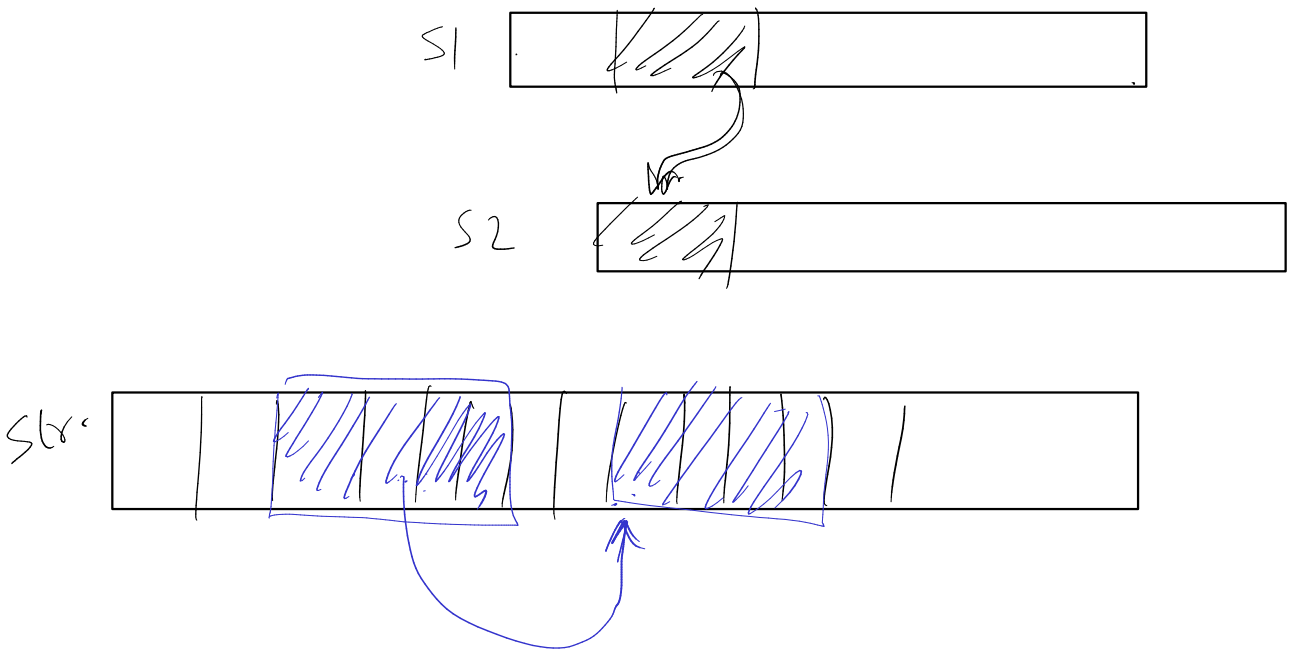


strcpy, strcat, strncpy etc....

IMP should be used on Arrays – NOT- overlapping-Area.

memmove is SAFE to use in overlapped Area.



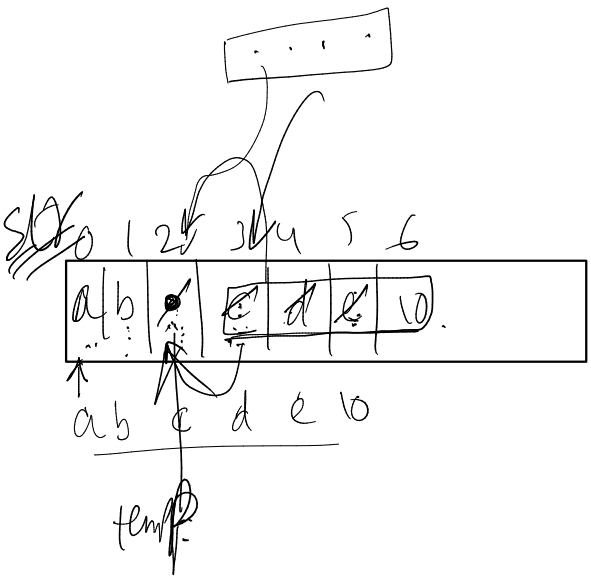

---

W.A.P to REMOVE, the 1st non-alphabet in a string

S1 => "VECTOR" => "VECTOR"

S1 => "1ab2c3" => "ab2c3"

S1 => "ab.cd" => "abcd."




```
char str[30]; char *temp;
puts("Enter a string:"); gets(str);
for(temp= str; *temp ; temp++) { if ( isalpha(*temp)==0)
                                    memmove(temp, temp+1, strlen(temp+1)+1)
}
```

- W.A.P to remove Extra blankspaces in a String.

str =) "␣␣␣abcd␣␣␣e␣fg␣␣"

- W.A.P to remove all Non-alphabets in a String

---

const char *p ;

VS char *p;